

Dzień 4

Security — pełen stack

Plan dnia

Pod-level hardening:

1. **Debug Pod** — kubectl debug, ephemeral containers
2. **Pod Security Admission + SecurityContext**
3. **Admission Controllers** + ValidatingAdmissionPolicy (CEL)
4. **HashiCorp Vault**

Cluster security tools:

5. **kube-bench** (CIS audit)
6. **Trivy Operator** (in-cluster scanning)
7. **Falco** (runtime detection)
8. **OPA / Gatekeeper** (policy as code)

Teoria: Service Mesh · Pentesty · Supply chain

→ Repo: `day4/`

Debug Pod (D4/01)

Distroless image = brak shella. Co teraz?

```
# Ephemeral container (in-place)
kubectl debug -it my-pod --image=nicolaka/netshoot \
  -- tcpdump -n port 8080

# Copy-to (klon z innym image)
kubectl debug -it my-pod --image=ubuntu \
  --share-processes --copy-to=my-pod-debug
```

→ `day4/01_debug_pod/`

Pod Security Admission (D4/02)

3 standardy × 3 modes per namespace:

Standard	Co
privileged	wszystko (default)
baseline	minimal restrictions (no host network/PID)
restricted	hardened (non-root, drop capabilities, seccomp)

```
apiVersion: v1
kind: Namespace
metadata:
  labels:
    pod-security.kubernetes.io/enforce: restricted
    pod-security.kubernetes.io/audit: restricted
    pod-security.kubernetes.io/warn: restricted
```

→ day4/02_psa_security/

Admission Controllers (D4/03)

Request → AuthN → AuthZ → Mutating Admission → Validating Admission → etcd

Trzy podejścia:

1. **Built-in** (PSA, ResourceQuota, LimitRanger, ...)
2. **Webhooks** (Vault, Gatekeeper, Kyverno)
3. **ValidatingAdmissionPolicy + CEL** (K8s 1.30+ stable, in-cluster, lekkie)

```
validations:  
  - expression: "object.spec.replicas >= 2"  
    message: "Min 2 repliki"
```

→ `day4/03_Admission_Controllers/`

HashiCorp Vault (D4/04)

Production secret management — alternatywa dla K8s Secrets.

3 wzorce integracji:

1. **CSI Driver** — secrets mounted jako pliki (`csisecret.yaml`)
2. **CSI + env** — secrets jako env vars
3. **Vault Agent Injector** — sidecar refreshujący secrets

```
helm install vault hashicorp/vault --version 0.24.1
```

→ `day4/04_vault/`

SecurityContext (D4/05)

```
spec:
  securityContext:
    runAsNonRoot: true
    runAsUser: 1000
    seccompProfile: { type: RuntimeDefault }
  containers:
    - securityContext:
        allowPrivilegeEscalation: false
        readOnlyRootFilesystem: true
        capabilities: { drop: ["ALL"] }
```

Defense in depth: image hardening + SecurityContext + PSA + OPA

→ `day4/05_security_context/`

kube-bench — CIS audit (D4/06)

```
kubectl apply -f kube-bench-job.yaml
kubectl logs job/kube-bench
# [PASS] 1.1.1 ...
# [FAIL] 5.1.5 default ServiceAccounts not actively used
# [WARN] 5.1.6 ...
```

CIS Kubernetes Benchmark = ~150 best practices security

- Sekcja 1-3: Control Plane
- Sekcja 4: Worker
- Sekcja 5: Policies (RBAC, PSA, NetworkPolicy)

→ `day4/06_kube_bench/`

Trivy Operator (D4/07)

Ciągłe skanowanie obrazów uruchomionych Pod-ów.

```
helm install trivy-operator aqua/trivy-operator -n trivy-system
kubectl get vulnerabilityreports -A
kubectl get configauditreports -A
kubectl get rbacassessmentreports -A
kubectl get exposedsecretreports -A
```

Grafana dashboard 17813 — Trivy reports visualization

→ `day4/07_trivy_k8s/`

Falco — runtime security (D4/08)

eBPF-based runtime detection. Wykrywa:

- Shell w produkcji
- Zapis do `/etc/`
- Mount sensitive paths
- Nieoczekiwany egress
- ECC errors / hardware events

```
- rule: Custom - Write to /etc
  condition: evt.is_open_write and fd.name startswith /etc/ and container
  output: "Zapis do /etc (user=%user.name file=%fd.name container=%container.name)"
  priority: WARNING
```

Falcosidekick → Slack / PagerDuty / Elastic / Kafka

→ `day4/08_falco/`

OPA / Gatekeeper (D4/09) — ConstraintTemplate

Policy as code (Rego). Najpierw definiujemy klasę policy:

```
kind: ConstraintTemplate
spec:
  crd: { spec: { names: { kind: K8sRequiredLabels } } }
  targets:
    - rego: |
        violation[{"msg": msg}] {
          missing := required - provided
          count(missing) > 0
          msg := sprintf("missing labels: %v", [missing])
        }
```

ConstraintTemplate generuje **nowy CRD** (`K8sRequiredLabels`) — meta-CRD pattern.

OPA / Gatekeeper — Constraint (instancja)

Z ConstraintTemplate wygenerowanego CRD tworzymy konkretną politykę:

```
kind: K8sRequiredLabels
spec:
  match: { kinds: [{ apiGroups: [""], kinds: [Pod] }] }
  parameters: { labels: ["owner"] }
```

Każdy Pod bez labela `owner` → admission webhook deny.

Alternatywa: Kyverno (YAML zamiast Rego, prostsza dla większości use cases)

→ `day4/09_opa_gatekeeper/`

Service Mesh — kiedy? (D4/10)

⚠ Tylko teoria, nie ćwiczenie

	Istio	Linkerd	Cilium SM
Sidecar	Envoy	linkerd2-proxy (Rust)	eBPF + Envoy L7
Złożoność	wysoka	niska	średnia
Latency overhead	5-10ms	1-3ms	minimal

Kiedy: 50+ microservices, zero-trust, mTLS compliance

NIE: monolit + 2-3 services, brak platform team

→ `day4/10_service_mesh_intro.md`

Pentesty K8s (D4/11)

⚠ Tylko teoria + linki

Faza 1: Recon (kube-hunter)

Faza 2: Compromised Pod → SA token, Kubelet API

Faza 3: Lateral movement (brak NetworkPolicy)

Faza 4: Persistence (mutating webhook, hidden CronJob)

Narzędzia: kube-hunter, peirates, amicontained, kubescape

Frameworks: MITRE ATT&CK Containers, NSA/CISA K8s Hardening Guide

→ `day4/11_cluster_pentest_intro.md`

Supply Chain (D4/12)

Cross-link D1 (Cosign + SBOM + Trivy) → D4 (admission policy):

```
git → CI build → Trivy scan → Cosign sign + SBOM → registry
                        ↓
                Sigstore Policy Controller
                        ↓
                K8s admission accept/deny
                        ↓
                Trivy Operator runtime scan
```

SLSA framework: L1-L4 dojrzałości supply chain

→ `day4/12_supply_chain.md`

Podsumowanie D4

- ✓ PSA + SecurityContext (Pod-level hardening)
- ✓ Vault (production secrets)
- ✓ kube-bench (compliance audit)
- ✓ Trivy Operator (runtime CVE scan)
- ✓ Falco (anomaly detection)
- ✓ OPA/Gatekeeper (policy enforcement)
- ✓ Service Mesh, Pentesty, Supply Chain (theory)

Jutro: Helm, monitoring, kubeadm, **AI/GPU na K8s**